序号（学号）： 227020114

长春大学旅游学院
毕 业 设 计（论 文）

基于 SpringBoot 与智能推荐的家政服务匹配平台的设计与实现

姓　　名	齐彦磊
学　　院	人工智能学院
专　　业	物联网工程
班　　级	2270201
指导教师	邹津婷（副教授）

2026 年 3 月 17 日

基于 SpringBoot 与智能推荐的家政服务匹配平台的设计与实现

[摘 要] 当下社会生活节奏持续加快，人口老龄化程度不断加深，家政服务已经从可选消费转变为家庭生活刚需，市场需求持续攀升。传统家政服务平台普遍存在供需匹配不精准、本地服务筛选繁琐、用户挑选效率低下等痛点，大量外地无关服务占用展示资源，用户需手动筛选才能找到适配的本地服务，整体使用体验较差，供需双方对接效率难以提升。针对这一系列行业痛点，本文遵循软件工程标准开发流程，设计并实现一套基于 SpringBoot 与轻量化智能推荐的家政服务匹配平台。本系统采用 Java 语言为核心开发语言，后端基于 SpringBoot 框架搭建，搭配 Spring Cloud Alibaba 微服务架构实现模块解耦，依托 Maven 完成项目依赖管理，选用 MySQL 数据库存储用户信息、服务信息、订单数据等核心业务数据，通过 Redis 缓存实现高频数据优化，提升系统响应速度。前端采用 Vue3 框架搭建可视化界面，实现前后端分离开发与运行。系统核心实现用户定位、本地服务自动筛选、智能推荐排序、全流程业务管控四大核心功能，细分用户管理、服务管理、预约管理、订单管理、权限管控等模块，覆盖普通用户、家政服务人员、平台管理员三类角色的全部操作需求，解决传统家政平台的核心痛点。系统开发完成后，通过功能测试与性能测试开展全面验证，测试结果表明，系统核心功能运行稳定，本地服务筛选准确率高、接口响应速度快，界面操作简洁易用，能够有效实现家政供需双方的高效对接，完全满足中小型家政平台的使用需求。

[关键词] 家政服务平台　智能推荐　本地服务匹配

装

订

线

Design and implementation of housekeeping service matching platform based on SpringBoot and intelligent recommendation

[Abstract] Nowadays, the pace of social life continues to accelerate, and the degree of population aging is deepening. Housekeeping services have changed from optional consumption to a rigid demand for family life, and the market demand continues to rise. Traditional housekeeping service platforms generally have pain points such as inaccurate supply-demand matching, cumbersome local service screening, and low user selection efficiency. A large number of irrelevant services from other places occupy display resources, and users need to manually screen to find suitable local services, resulting in poor overall user experience and difficulty in improving the docking efficiency between supply and demand parties. In response to this series of industry pain points, this paper designs and implements a housekeeping service matching platform based on SpringBoot and lightweight intelligent recommendation in accordance with the standard software engineering development process.The system uses Java as the core development language, the backend is built based on the SpringBoot framework, and the Spring Cloud Alibaba microservice architecture is used to achieve module decoupling. It relies on Maven to complete project dependency management, selects MySQL database to store core business data such as user information, service information and order data, and optimizes high-frequency data through Redis cache to improve system response speed. The frontend uses the Vue3 framework to build a visual interface, realizing separate front-end and back-end development and operation. The core of the system realizes four core functions: user positioning, automatic local service screening, intelligent recommendation sorting, and full-process business control. It subdivides modules such as user management, service management, appointment management, order management, and authority control, covering all operation needs of three roles: ordinary users, housekeeping service personnel, and platform administrators, and solves the core pain points of traditional housekeeping platforms.After the system development is completed, a comprehensive verification is carried out through functional testing and performance testing. The test results show that the core functions of the system operate stably, the local service screening accuracy is high, the interface response speed is fast, and the interface operation is simple and easy to use. It can effectively realize efficient docking between housekeeping supply and

demand parties, and fully meets the use needs of small and medium-sized housekeeping platforms.

[Keywords]HousekeepingServicePlatform Microservice Intelligent Recommendation Local Service Matching

装

订

线

目　　录

中文摘要.....I
Abstract.....II
引言.....1
第 1 章　绪论.....1
1.1 课题背景及研究目的和意义.....1
1.2 家政平台与相关技术发展概况.....1
1.3 主要研究内容.....2
1.4 论文结构.....2
第 2 章　相关技术与开发环境.....3
2.1 核心技术概述.....3
2.2 系统开发与运行环境.....4
第 3 章　系统需求分析.....4
3.1 系统可行性分析.....4
3.2 系统功能需求分析.....5
3.3 非功能性需求分析.....6
3.4 核心业务流程分析.....6
第 4 章　系统总体设计.....9
4.1 系统总体设计.....9
4.2 数据库设计.....11
4.3 核心业务流程设计.....16
第 5 章　系统详细设计与实现.....19
5.1 系统工程文件结构.....19
5.2 核心功能详细实现.....19
5.3 数据库操作实现.....20
5.4 微服务通信实现.....20
5.5 系统核心界面实现.....20
第 6 章　系统测试.....20
6.1 测试目的与方法.....20
6.2 测试环境与测试用例.....21
6.3 测试结果与分析.....21
第 7 章　结论与展望.....22
7.1 论文工作总结.....22
7.2 系统不足与未来展望.....22
参考文献.....23

装

订

线

基于 SpringBoot 与智能推荐的家政服务匹配平台设计

第 1 章 绪论

1.1 课题背景及研究目的和意义

随着国民生活水平的提升以及生活节奏的不断加快，日常保洁、育儿护理、养老照料等家政服务需求持续上涨，家政服务行业市场规模逐年扩大，逐渐成为生活服务领域的重要组成部分。与此同时，人口老龄化趋势加深，独居老人、双职工家庭数量增多，进一步推动了家政服务的刚需化发展，用户对家政服务的便捷性、精准性、时效性要求也越来越高。

传统家政服务模式多依赖线下中介、熟人介绍，信息流通闭塞，服务范围受限；线上传统家政平台则存在明显短板，平台未针对本地服务做专项优化，大量外地服务充斥页面，用户需要反复筛选才能找到所在区域的可用服务，耗时费力。同时，传统平台服务排序逻辑单一，缺乏针对性推荐，供需双方匹配效率低下，用户挑选成本高，服务人员也难以获得精准客源，行业整体运营效率偏低。

在技术层面，SpringBoot 微服务架构、Redis 缓存、前端 Vue 框架等技术日趋成熟，开发门槛降低，为轻量化生活服务平台的开发提供了完善的技术支撑。基于这样的行业背景与技术环境，本文开展基于 SpringBoot 与智能推荐的家政服务匹配平台的设计与实现工作，旨在打造一款轻量化、易操作、高效率的家政服务平台，解决传统模式的核心痛点，实现本地家政服务的精准匹配。

本课题的研究具备较强的现实意义与实践意义，现实层面能够为用户提供便捷的本地家政服务查找渠道，为服务人员提供精准的客源曝光，提升行业整体对接效率；实践层面能够将所学软件工程知识落地应用，完整覆盖项目开发全流程，符合本科毕业设计的考核要求，也为同类小型生活服务平台开发提供参考范例。

1.2 家政平台与相关技术发展概况

国内家政服务平台发展主要分为三个阶段，早期以线下中介为主，信息不对称问题严重，服务质量难以保障；中期线上平台初步兴起，实现服务信息线上化展示，但功能单一，缺乏本地化与智能化设计；现阶段头部平台逐步实现智能化、本地化升级，但中小型家政平台仍存在技术落后、功能不完善的问题，相关轻量化、低成本的平台开发方案较为稀缺。

在技术发展方面，SpringBoot 框架凭借自动配置、内嵌服务器、简化部署等优势，成为 Java 后端开发的主流框架，广泛应用于各类中小型项目开发；微服务架构通过模块解耦，提升了系统的可扩展性与可维护性，适配平台后续功能升级；Redis 缓存技术则有效解决了高频数据查询的性能瓶颈，大幅提升系统响应速度。这些成熟技术的普及，为本系统的开发提供了坚实的技术基础，也让轻量化、低成本的家政平台开发成为可能。

在智能推荐领域，针对生活服务类平台的轻量化推荐算法日趋成熟，无需复杂的深度学习模型，通过热度、距离、价格等基础维度加权排序，即可满足

中小型平台的推荐需求，兼顾运行效率与开发成本，这也是本系统选用轻量化推荐方案的核心依据。

1.3 主要研究内容

本课题严格遵循软件工程开发流程，围绕家政服务匹配平台的设计与实现，从需求分析、总体设计、详细开发、系统测试四个核心环节开展研究，具体研究内容如下：

（1）开展全面的需求调研，梳理三类用户角色的核心需求，从经济、技术、法律、运行四个维度完成可行性分析，明确系统功能边界与非功能性要求，确定系统整体开发方向。

（2）完成系统总体设计，采用前后端分离+SpringBoot 框架，划分核心功能模块，明确各模块职责与调用关系；设计系统数据库，完成核心数据表的概念设计、逻辑设计与物理设计，规范数据存储结构。

（3）基于 SpringBoot 框架完成后端代码开发，依托 Vue3 实现前端页面搭建，实现用户注册登录、定位获取、本地服务筛选、智能推荐排序、服务预约、订单管理、权限管控等核心功能；通过 Redis 缓存优化系统性能，通过 Spring Cloud 实现微服务通信与管理。

（4）对系统进行全面测试，设计功能测试用例与性能测试方案，验证系统核心功能的完整性、运行稳定性与响应速度，排查功能漏洞与性能问题，完成优化整改。

（5）总结课题研究成果，梳理系统开发过程中的难点与解决方案，分析系统现存不足，提出后续优化与功能拓展方向，完成毕业论文的撰写与整理。

1.4 论文结构

第 1 章，主要阐述课题研究背景、目的与意义，梳理家政平台与相关技术的发展现状，明确本文主要研究内容与论文整体结构。

第 2 章，对本课题所使用到的相关技术，以及开发环境的介绍

第 3 章，系统需求分析，开展可行性分析与需求拆解，划分用户角色，明确功能需求与非功能性需求，梳理核心业务流程，确定系统开发的核心目标。

第 4 章，系统总体设计，制定系统设计原则，完成系统架构、功能模块、数据库、核心业务流程的整体设计，搭建系统整体开发框架。其中包括了系统的运行环境、系统总体设计、和系统的数据库概念设计即 E-R 图、数据库逻辑结构设计、物理设计。

第 5 章，系统详细设计与实现，介绍系统工程文件结构，详解各核心模块的代码实现逻辑、功能落地方式，展示系统核心界面效果，说明关键技术问题的解决方案。

第 6 章，系统测试，明确测试目的、方法与环境，设计测试用例开展功能与性能测试，统计测试结果并分析，验证系统是否达到设计要求。

第 7 章，结论与展望，总结全文研究工作与系统开发成果，分析系统现有不足，提出未来优化与功能拓展方向。

最后为参考文献与致谢，梳理课题研究与论文撰写过程中参考的相关文献资料，感谢指导教师与学校提供的支持与帮助。

第 2 章　相关技术与开发环境

2.1 核心技术概述

2.1.1 SpringBoot 框架

SpringBoot 是基于 Spring 框架衍生的轻量化后端开发框架，核心优势是简化 Spring 框架的复杂配置，实现“约定大于配置”的开发理念。框架支持内嵌 Tomcat、Jetty 等服务器，无需额外部署独立服务器，项目可直接运行；通过自动配置机制，无需手动编写大量 XML 配置文件，大幅缩短开发周期；同时支持各类第三方依赖的快速集成，兼容性强，是当前 Java Web 项目开发的首选框架，完全适配本系统的轻量化开发需求。

2.1.2 Spring Cloud Alibaba 微服务组件

微服务架构是将单体系统拆分为多个独立、低耦合的小型服务模块，各模块独立运行、独立部署，通过接口实现数据交互。Spring Cloud Alibaba 是国内主流的微服务解决方案，本系统主要用到其中 Nacos 与 OpenFeign 两大核心组件，Nacos 负责微服务的注册、发现与管理，统一监控各服务运行状态；OpenFeign 负责实现微服务之间的高效通信，简化服务调用代码，实现模块间的无缝对接，提升系统的可扩展性与可维护性。

2.1.3 MySQL 与 Redis 数据库

MySQL 是一款开源关系型数据库，具备数据存储稳定、查询效率高、支持事务管理等优势，适合存储用户信息、服务信息、订单数据等结构化核心业务数据，完全满足本系统的数据存储需求，且开源免费，降低项目开发成本。

Redis 是一款高性能内存型缓存数据库，数据读写速度远高于传统磁盘型数据库，本系统将高频访问的本地服务数据、热门服务列表、用户定位信息等存入 Redis，设置合理过期时间，用户发起查询时优先从缓存读取数据，既减轻 MySQL 数据库的访问压力，又大幅提升系统响应速度，解决高并发查询的性能瓶颈。

2.1.4 Vue3 前端框架

Vue3 是一款轻量级前端渐进式框架，采用组件化开发模式，页面组件可重复复用，降低前端开发工作量；双向数据 binding 机制让页面数据与操作实时同步，提升用户交互体验；框架学习成本低、运行效率高，适配各类管理系统与服务平台的前端页面开发。本系统采用 Vue3 搭建可视化操作界面，实现前后端分离开发，前后端通过接口对接，互不干扰，提升开发效率。

2.1.5 其他核心技术

MyBatis-Plus 是 MyBatis 的增强工具，简化数据库 CRUD 操作，无需编写大量重复 SQL 语句，提升后端数据处理效率；MD5 加密技术用于用户密码加密存储，保障用户账号安全，避免明文密码泄露；Maven 项目管理工具用于统一

管理项目第三方依赖，自动完成依赖下载与版本匹配，避免 jar 包冲突问题，规范项目结构。

2.2 系统开发与运行环境

2.2.1 开发工具

后端开发选用 IntelliJ IDEA 作为核心开发工具，前端开发选用 VS Code，数据库管理选用 Navicat，项目依赖管理通过 Maven 完成，版本控制依托 Git 实现，整套工具组合适配 Java 前后端项目开发，操作便捷、功能完善。

2.2.2 软硬件运行环境

硬件环境：普通办公电脑即可，配置要求为 CPU i3 及以上、内存 8G 及以上、硬盘剩余空间 50G 及以上，无需专用服务器，降低部署与运行成本。

软件环境：操作系统支持 Windows 10 及以上版本，JDK 版本为 1.8，MySQL 版本为 8.0，Redis 版本为 5.0 及以上，浏览器支持 Chrome、Edge 等主流浏览器，环境搭建简单、适配性强。

第 3 章 系统需求分析

3.1 系统可行性分析

3.1.1 经济可行性

本系统开发全程采用开源技术与工具，SpringBoot、Vue3、MySQL、Redis 等核心技术均为开源免费使用，无版权费用与软件采购成本；开发与运行无需高端硬件设备，普通电脑即可完成开发与部署，整体开发成本极低。系统上线后可有效提升家政服务对接效率，降低人工管理成本，具备良好的经济可行性。

3.1.2 技术可行性

本系统选用的 SpringBoot、微服务、Redis 等技术均为行业成熟技术，相关技术文档完善、教程丰富，开发难度适中；本人在校期间已系统学习 Java 编程、数据库、前后端开发等相关知识，具备完整的项目开发能力，能够独立完成系统设计与实现工作，技术层面完全可行。

3.1.3 法律可行性

本系统仅用于家政服务信息匹配与管理，全程遵守网络安全与数据保护相关法律法规，用户数据仅用于平台业务运行，不泄露、不滥用第三方信息；所有技术均采用开源合规版本，无侵权行为，具备完全的法律可行性。

3.1.4 运行可行性

系统界面设计简洁直观，操作逻辑符合普通用户的使用习惯，无需专业培训即可快速上手；系统兼容性强，支持主流浏览器访问，部署流程简单，适合

装 订 线

中小型家政企业与个人使用，运行可行性良好。

3.2 系统功能需求分析

3.2.1 系统总体目标

本系统旨在打造一款轻量化、本地化、智能化的家政服务匹配平台，实现用户快速查找本地家政服务、服务人员精准对接客源、管理员高效管控平台的核心目标，解决传统家政平台匹配不精准、筛选繁琐的痛点，提升家政服务供需对接效率，界面简洁、运行稳定、功能实用，满足日常使用需求。

3.2.2 用户角色划分

本系统共划分三类用户角色，各类角色权限清晰、功能分工明确，覆盖家政服务全业务流程：

（1）普通用户：主要为有家政服务需求的人群，可完成注册登录、定位授权、浏览本地家政服务、筛选服务类型、预约服务、查看个人订单与预约记录、修改个人信息等操作。

（2）家政服务人员：主要为提供家政服务的从业者，可完成注册认证、发布个人服务信息、编辑下架服务、查看用户预约申请、管理个人订单、更新服务状态等操作。

（3）平台管理员：负责平台整体运营管理，可完成用户账号管理、服务信息审核、异常订单处理、数据统计查看、权限分配等操作，保障平台正常稳定运行。系统整体用例图如图 3-1 所示。

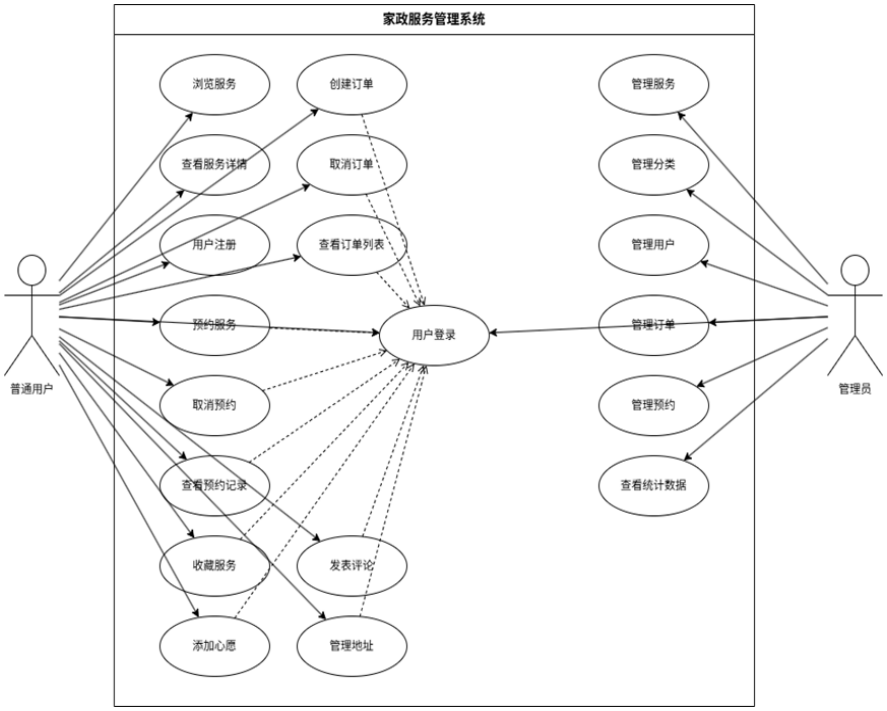

图 3-1 系统整体用例图

3.2.3 核心功能模块需求

（1）用户管理模块：涵盖用户注册、登录、密码加密、信息修改、角色权限管控、账号状态管理功能，区分三类角色操作权限，保障账号安全。

（2）服务管理模块：支持家政服务人员发布、编辑、删除、上下架服务信息，填写服务名称、价格、内容、所在城市等详情，管理员可审核服务信息，屏蔽违规内容。

（3）本地服务模块：通过用户定位或手动选择城市，自动筛选当前城市的上架服务，屏蔽外地无关服务，实现本地服务优先展示，减少用户筛选成本。

（4）智能推荐模块：结合服务热度、距离远近、价格合理性三个维度，对本地服务进行加权排序，将优质、适配的服务优先展示给用户，提升匹配效率。

（5）预约管理模块：用户可选择服务预约日期与时间段，系统校验预约冲突，提交预约申请，服务人员可查看并确认预约，支持预约状态更新。

（6）订单管理模块：预约成功后自动生成对应订单，记录订单编号、服务信息、订单状态、下单时间等详情，支持三类角色按权限查看订单，管理员可处理异常订单。

（7）系统管理模块：仅管理员可使用，涵盖用户管理、服务审核、数据统计、系统参数配置等功能，实现平台全方位管控。

3.3 非功能性需求分析

（1）稳定性需求：系统核心功能需 7×24 小时稳定运行，无卡顿、崩溃、数据错乱等问题，支持多用户同时在线操作，保障业务正常开展。

（2）性能需求：核心接口响应时间控制在 1 秒以内，页面加载流畅，高频数据查询通过 Redis 优化，避免长时间等待，提升用户体验。

（3）易用性需求：界面布局清晰、按钮标识明确，操作流程简洁，无复杂步骤，普通用户无需培训即可快速上手，降低使用门槛。

（4）安全性需求：用户密码采用 MD5 加密存储，接口设置权限校验，禁止未授权访问，用户数据加密存储，防止信息泄露与非法篡改。

（5）可扩展性需求：系统采用微服务模块化设计，后续新增服务类型、支付功能、评价体系等拓展功能时，不影响原有模块正常运行。

3.4 核心业务流程分析

普通用户核心业务流程：用户打开平台→注册/登录账号→系统获取用户定位→自动展示本地服务→用户浏览推荐服务→选择心仪服务→提交预约申请→系统校验预约信息→生成预约记录与订单→服务人员确认预约→用户按约定享受服务→订单完成。普通用户核心业务活动图如图 3-2 所示。

家政服务人员核心业务流程：服务人员注册认证→登录平台→发布个人服务信息→填写城市与服务详情→管理员审核通过→服务上架展示→接收用户预约申请→确认/拒绝预约→更新订单状态→服务完成→订单归档。服务发布与管理流程如图 3-3 所示。用户注册登录流程如图 3-5 所示。

管理员核心业务流程：管理员登录后台→审核服务信息→管理用户账号→查看全平台订单→处理异常订单→统计平台数据→维护系统正常运行。订单管理流程如图 3-4 所示。

装 订 线

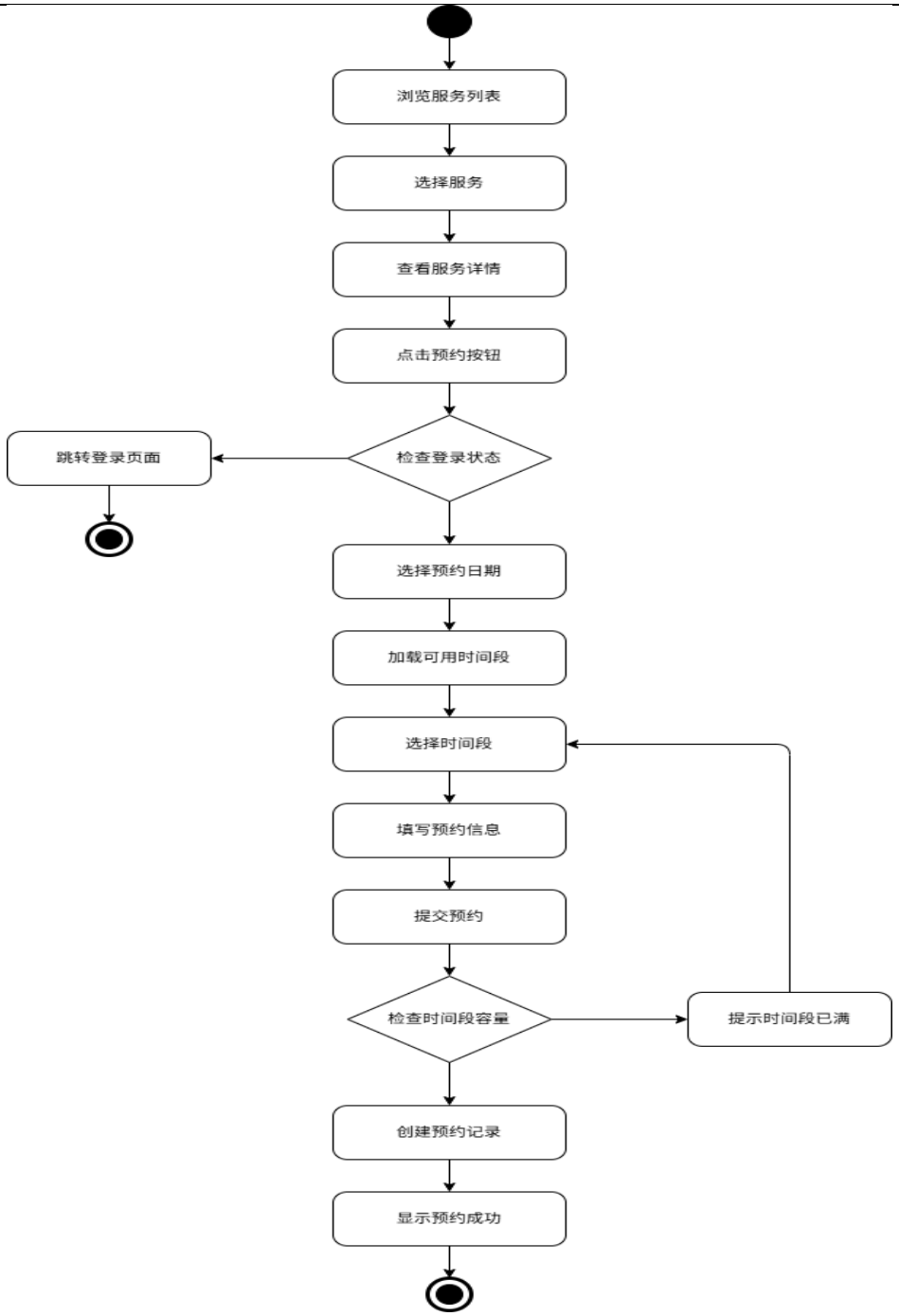

图 3-2 普通用户业务活动图

装

订

线

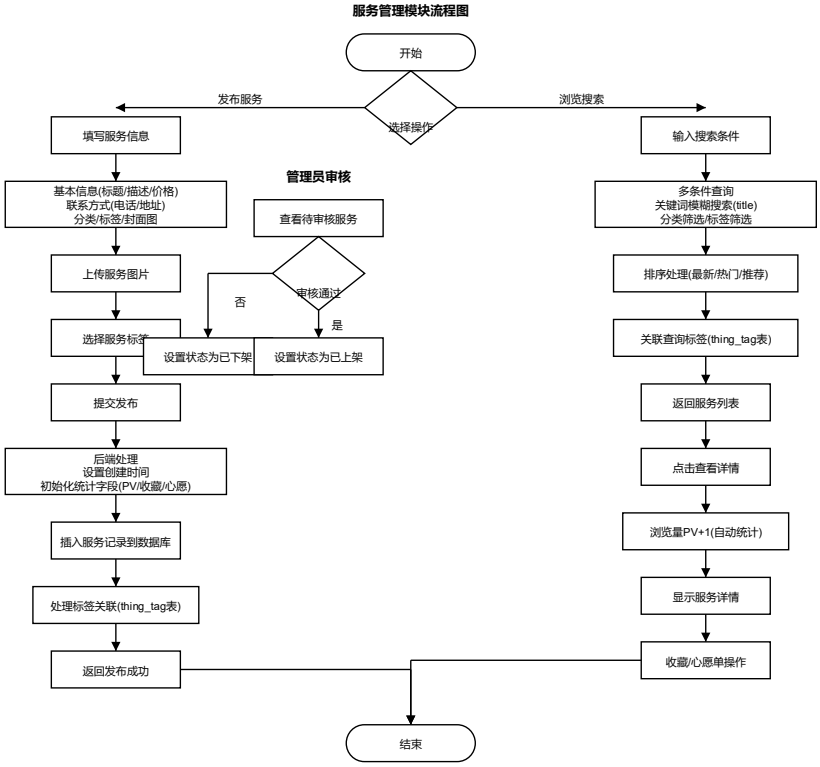

图 3-3 服务发布与管理流程图

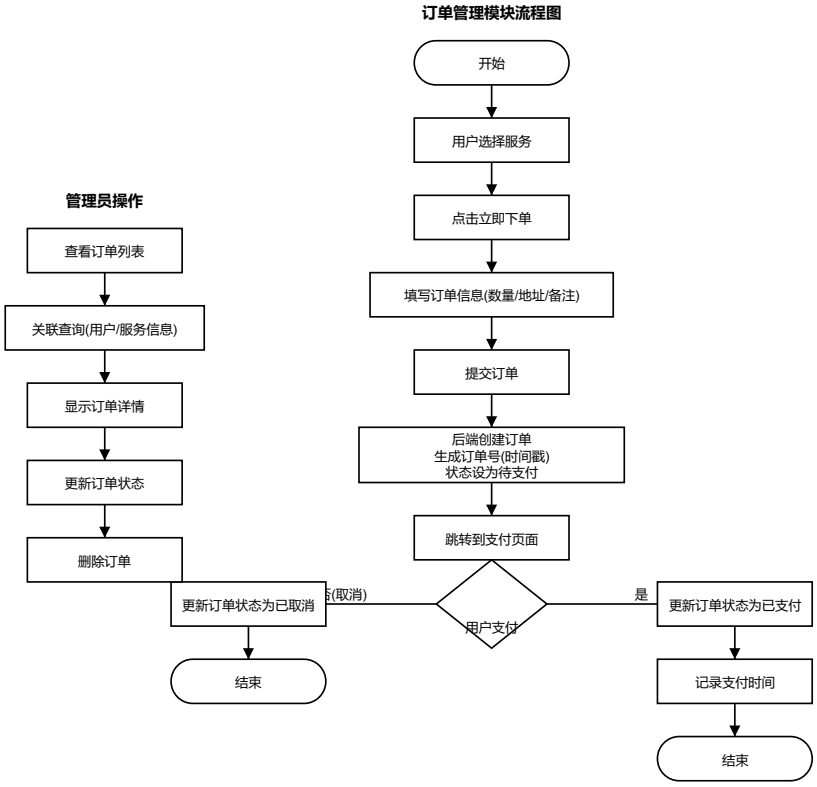

图 3-4 订单管理流程图

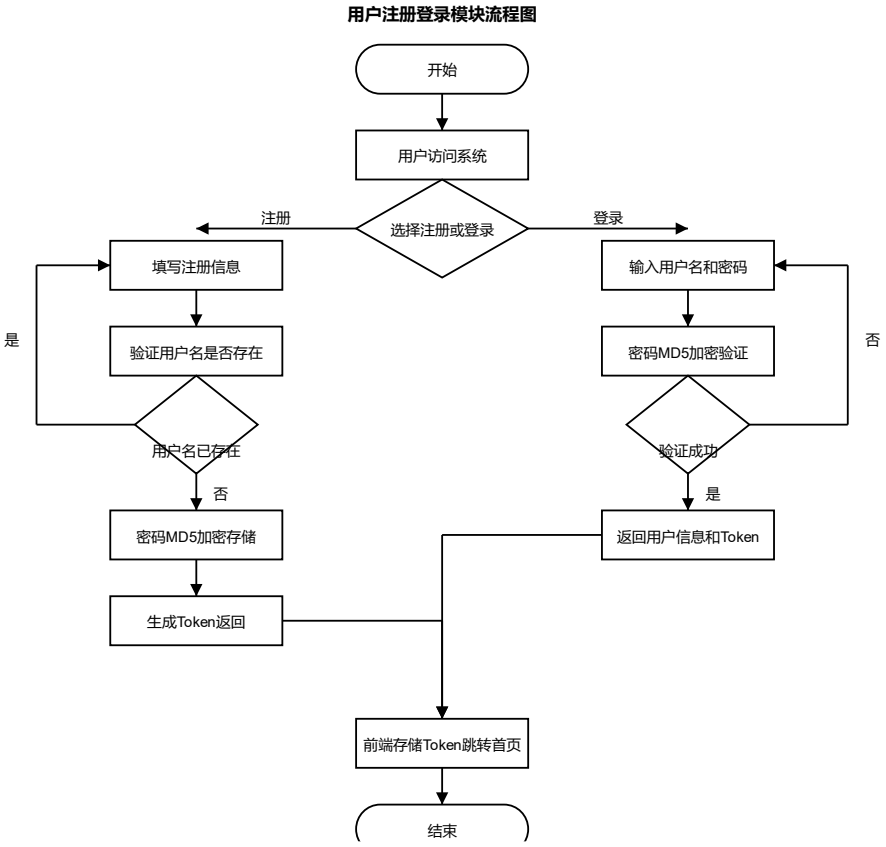

图 3-5 用户注册登录模块图

第 4 章　系统总体设计

4.1 系统总体设计

4.1.1 系统总体设计原则

本系统遵循实用性、轻量化、稳定性、安全性、可扩展性五大设计原则，优先满足核心业务需求，不做冗余功能设计；采用成熟轻量化技术，降低开发与部署成本；保障系统运行稳定、数据安全；模块设计低耦合，方便后续功能升级与维护。

4.1.2 系统架构设计

本系统采用前后端分离+微服务分层架构，自上而下分为四层，各层职责清晰、数据交互规范，保障系统高效稳定运行：

（1）前端展示层：基于 Vue3 框架搭建，负责页面渲染、用户交互操作，接收用户指令并向后端发送请求，展示后端返回的数据，包含登录注册页、首页、服务列表页、预约页、个人中心页、管理后台页等。

（2）网关与微服务层：核心包含 Nacos 服务注册中心、用户服务、服务管理服务、订单预约服务三大微服务，各微服务独立运行，通过 OpenFeign 实现相互通信，Nacos 统一管理服务状态，实现请求路由与负载均衡。

（3）数据持久层：负责连接数据库与缓存，处理数据增删改查操作，通过 MyBatis-Plus 实现与 MySQL 的数据交互，通过 Redis 工具类实现缓存数据的读写操作。

（4）数据存储层：包含 MySQL 业务数据库与 Redis 缓存数据库，MySQL 存储持久化业务数据，Redis 存储高频访问数据，两者分工协作，保障数据存储与查询效率。系统总体架构如图 4-1 所示。

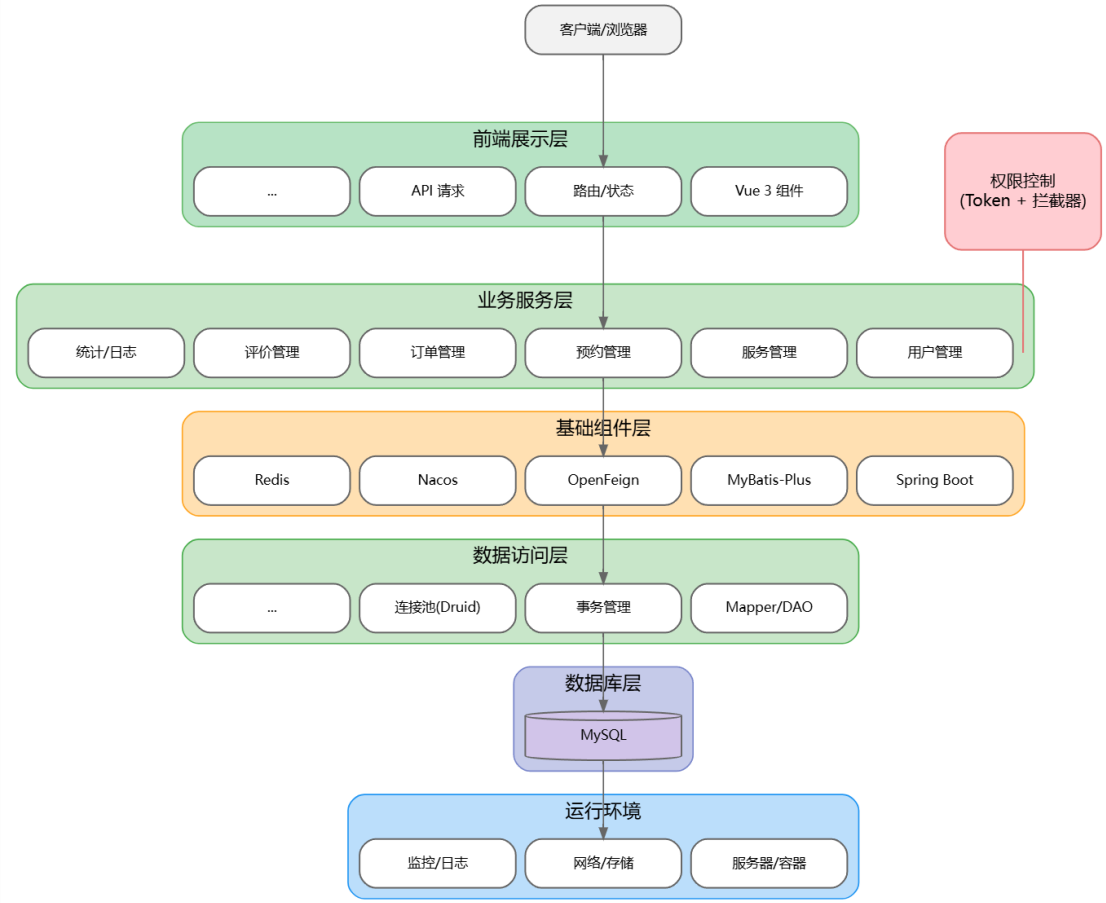

图 4-1 系统架构图

4.1.3 功能模块结构设计

本系统整体划分为七大核心功能模块，模块间低耦合、高内聚，通过接口实现数据交互，具体模块划分如下：

（1）用户管理模块：子功能包括用户注册、登录、密码修改、信息管理、权限控制、账号状态管理。

（2）服务管理模块：子功能包括服务发布、编辑、删除、上下架、信息审核、分类管理。

（3）本地服务模块：子功能包括定位获取、城市选择、本地服务筛选、外地服务屏蔽。

（4）智能推荐模块：子功能包括服务热度统计、距离计算、加权排序、推荐列表生成。

（5）预约管理模块：子功能包括时间段选择、预约提交、冲突校验、预约状态更新。

装 订 线

（6）订单管理模块：子功能包括订单生成、状态管控、订单查询、异常订单处理。

（7）系统管理模块：子功能包括用户管理、服务审核、数据统计、参数配置。系统功能模块划分如图 4-2 所示。

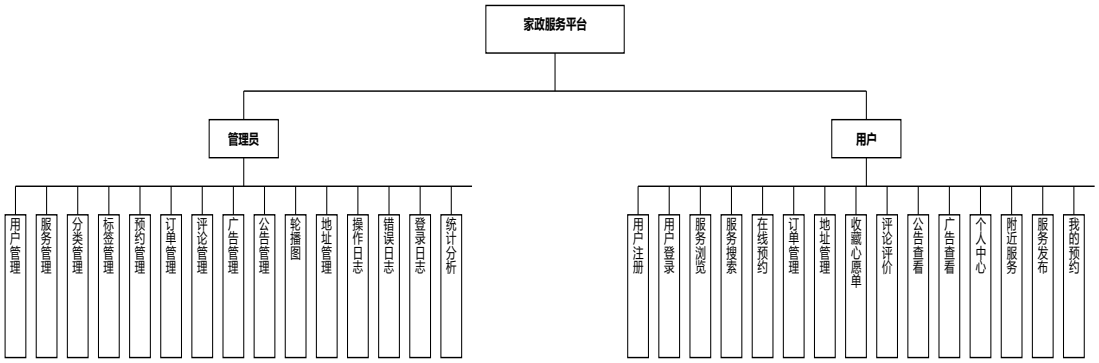

图 4-2 系统功能结构图

4.2 数据库设计

4.2.1 数据库概念结构设计（E-R 图）

数据库核心实体包括用户、家政服务、预约记录、订单四大类，实体间关联关系清晰：用户与家政服务为一对多关系，一个服务人员可发布多个服务；用户与预约为一对多关系，一个用户可发起多个预约；家政服务与预约为一对多关系，一个服务可被多个用户预约；预约与订单为一对一关系，一个有效预约对应生成一个订单。

用户实体核心属性：用户 ID、用户名、密码、昵称、手机号、角色类型、账号状态、创建时间。用户实体 E-R 图如图 4-3 所示。

家政服务实体核心属性：服务 ID、服务名称、服务描述、价格、所在城市、城市编码、服务状态、发布人 ID、浏览量、创建时间。服务实体 E-R 图如图 4-4 所示。

预约实体核心属性：预约 ID、预约编号、用户 ID、服务 ID、预约日期、时间段、预约状态、联系人、联系电话、创建时间。

订单实体核心属性：订单 ID、订单编号、预约 ID、用户 ID、服务 ID、订单状态、下单时间、更新时间。订单实体 E-R 图如图 4-5 所示。

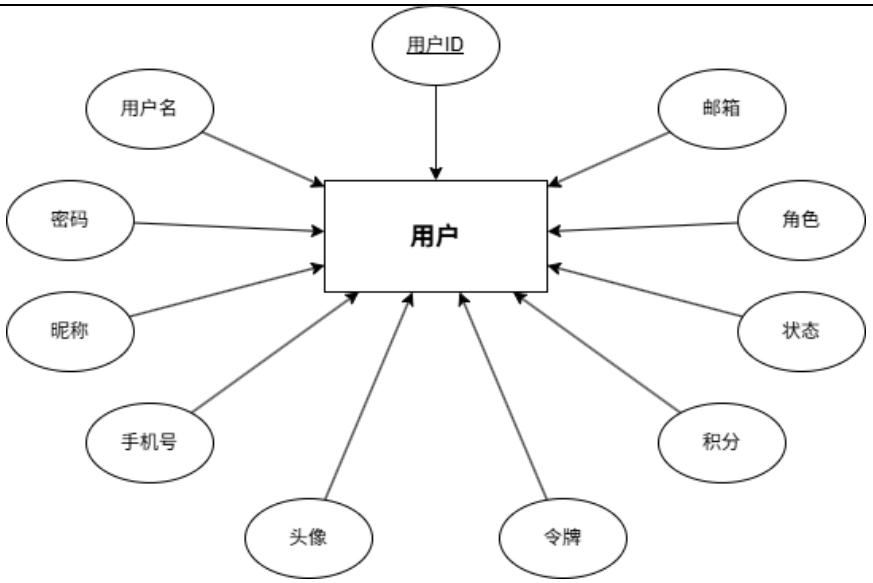

图 4-3 用户实体 E-R 图

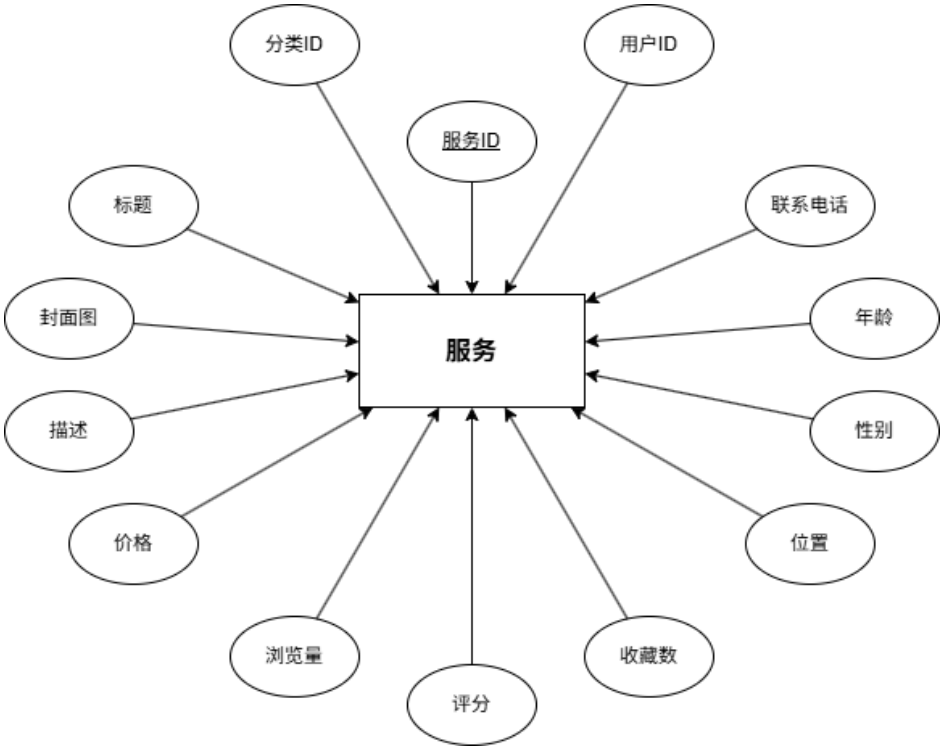

图 4-4 服务实体 E-R 图

装

订

线

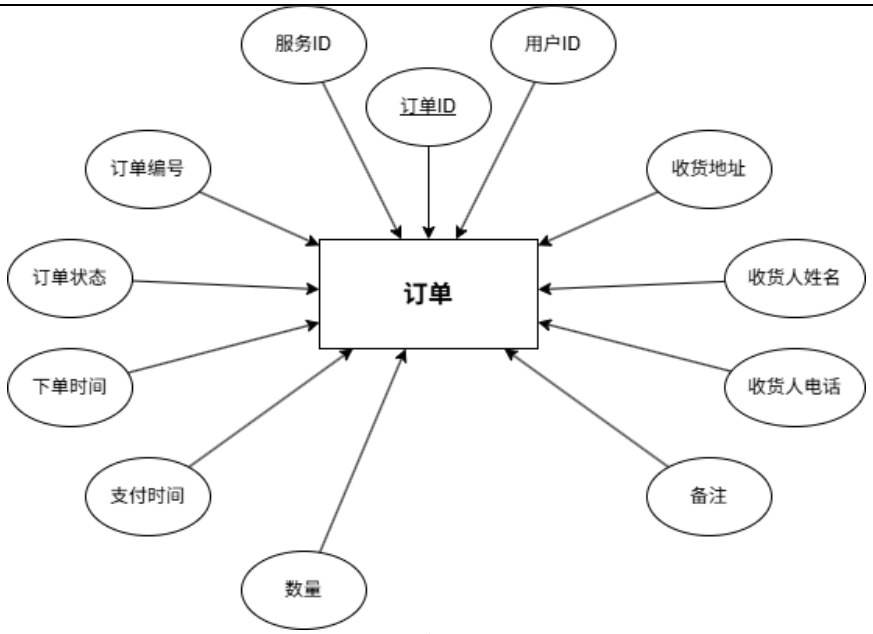

图 4-5 订单实体 E-R 图

4.2.2 数据库逻辑结构设计

将数据库实体转化为对应的数据表，规范字段名称、数据类型、约束条件，核心数据表如下：

（1）用户表（b_user）：存储所有用户账号信息，主键为用户 ID，字段包含 id（bigint，主键自增）、username（varchar，唯一非空）、password（varchar，非空）、nickname（varchar）、phone（varchar）、role（tinyint，0 普通用户/1 服务人员/2 管理员）、status（tinyint，0 正常/1 禁用）、create_time（varchar）。其表结构如表 4-1 所示

（2）家政服务表（b_thing）：存储服务信息，主键为服务 ID，字段包含 id（bigint，主键自增）、title（varchar，非空）、content（text）、price（varchar）、city（varchar）、city_code（varchar）、status（tinyint，0 上架/1 下架）、user_id（bigint，外键关联用户表）、views（int）、create_time（varchar）。其表结构如表 4-2 所示

（3）预约表（b_appointment）：存储预约信息，主键为预约 ID，字段包含 id（bigint，主键自增）、code（varchar，唯一）、user_id（bigint，外键关联用户表）、thing_id（bigint，外键关联服务表）、appoint_date（varchar）、time_id（bigint）、status（tinyint，0 待确认/1 已确认/2 已取消）、contact_name（varchar）、contact_phone（varchar）、create_time（varchar）。其表结构如表 4-3 所示

（4）订单表（b_order）：存储订单信息，主键为订单 ID，字段包含 id（bigint，主键自增）、code（varchar，唯一）、appoint_id（bigint，外键关联预约表）、user_id（bigint，外键关联用户表）、thing_id（bigint，外键关联服务表）、status（tinyint，0 待确认/1 已完成/2 已取消）、create_time（varchar）、update_time（varchar）。其表结构如表 4-4 所示。

表 4-1 用户表（b_user）结构设计

字段名	数据类型	长度	允许空	主键	说明
id	BIGINT	20	否	是	用户ID
username	VARCHAR	50	否	否	用户名（唯一）
password	VARCHAR	100	否	否	密码（加密存储）
role	VARCHAR	2	是	否	角色（1-管理员 2-用户 3-普通用户）
status	VARCHAR	1	是	否	状态（0-正常 1-禁用）
nickname	VARCHAR	20	是	否	昵称
avatar	VARCHAR	100	是	否	头像路径
mobile	VARCHAR	13	是	否	手机号
email	VARCHAR	50	是	否	邮箱
token	VARCHAR	200	是	否	登录令牌
create_time	DATETIME	-	是	否	创建时间

表 4-2 家政服务表（b_thing）结构设计

字段名	数据类型	长度	允许空	主键	说明
id	BIGINT	20	否	是	服务ID
title	VARCHAR	100	是	否	服务标题
classification_id	BIGINT	20	是	否	分类ID（外键）
description	TEXT	-	是	否	服务描述
cover	VARCHAR	100	是	否	封面图片路径
price	VARCHAR	50	是	否	服务价格
status	VARCHAR	1	是	否	状态（0-上架 1-下架）
wish_count	INT	11	是	否	心愿数量
collect_count	INT	11	是	否	收藏数量
create_time	DATETIME	-	是	否	创建时间

装 订 线

表 4-3 预约表（b_appointment）结构设计

字段名	数据类型	长度	允许空	主键	说明
id	BIGINT	20	否	是	预约ID
user_id	BIGINT	20	是	否	用户ID（外键）
thing_id	BIGINT	20	是	否	服务ID（外键）
appointment_date	DATE	-	是	否	预约日期
appointment_time	VARCHAR	20	是	否	预约时间段
status	VARCHAR	2	是	否	预约状态（1-待确认 2-已确认 3-已取消）
contact_name	VARCHAR	20	是	否	联系人姓名
contact_mobile	VARCHAR	13	是	否	联系人电话
address	VARCHAR	100	是	否	服务地址
remark	VARCHAR	200	是	否	预约备注
create_time	DATETIME	-	是	否	创建时间

表 4-4 订单表（b_order）结构设计

字段名	数据类型	长度	允许空	主键	说明
id	BIGINT	20	否	是	订单ID
order_number	VARCHAR	13	是	否	订单编号（唯一）
user_id	BIGINT	20	是	否	用户ID（外键）
thing_id	BIGINT	20	是	否	服务ID（外键）
count	INT	11	是	否	订单数量
status	VARCHAR	2	是	否	订单状态（1-待处理 2-已完成 3-已取消）
receiver_name	VARCHAR	20	是	否	收货人姓名
receiver_address	VARCHAR	100	是	否	收货地址
receiver_phone	VARCHAR	13	是	否	收货人电话
remark	VARCHAR	200	是	否	订单备注
create_time	DATETIME	-	是	否	创建时间

4.2.3 数据库物理结构设计

数据库命名规范统一，采用小写字母加下划线格式，字段数据类型适配业务需求，合理设置主键、外键、唯一约束与非空约束，建立常用查询字段索引，提升数据查询效率；MySQL 数据库编码采用 utf8mb4，支持特殊字符存储，Redis 缓存设置合理过期时间，避免缓存数据冗余占用内存。

4.3 核心业务流程设计

核心业务流程围绕用户找服务、预约服务、订单完成全流程设计，逻辑连贯、无冗余步骤，系统自动完成校验与数据处理，避免人工干预失误。用户预约服务时，系统自动校验用户权限、服务状态、时间段是否冲突，校验通过后生成预约与订单数据，同步更新对应状态；服务人员确认预约后，订单状态同步更新，全程数据实时同步，保障业务流程顺畅。

1. 注册登录流程设计

用户注册时，前端收集用户名与密码后提交至后端，系统首先校验参数完整性，随后查询数据库确认用户名唯一性，并比对两次密码输入是否一致。校验通过后，系统采用 MD5 加盐方式对密码进行加密存储，同时基于用户名与盐值生成唯一 Token，设置默认角色为普通用户（role=1）并写入数据库，完成注册。用户登录时，系统将前端传入的明文密码加密后与数据库记录比对，验证通过则返回包含 Token 的用户对象，前端将 Token 持久化至 localStorage，后续所有需要鉴权的请求均通过请求头携带 Token，由拦截器（AccessInterceptor）统一完成身份验证与权限级别校验，支持普通用户、演示账号、管理员三种角色的差异化访问控制。注册登录流程如图 4-3 所示。

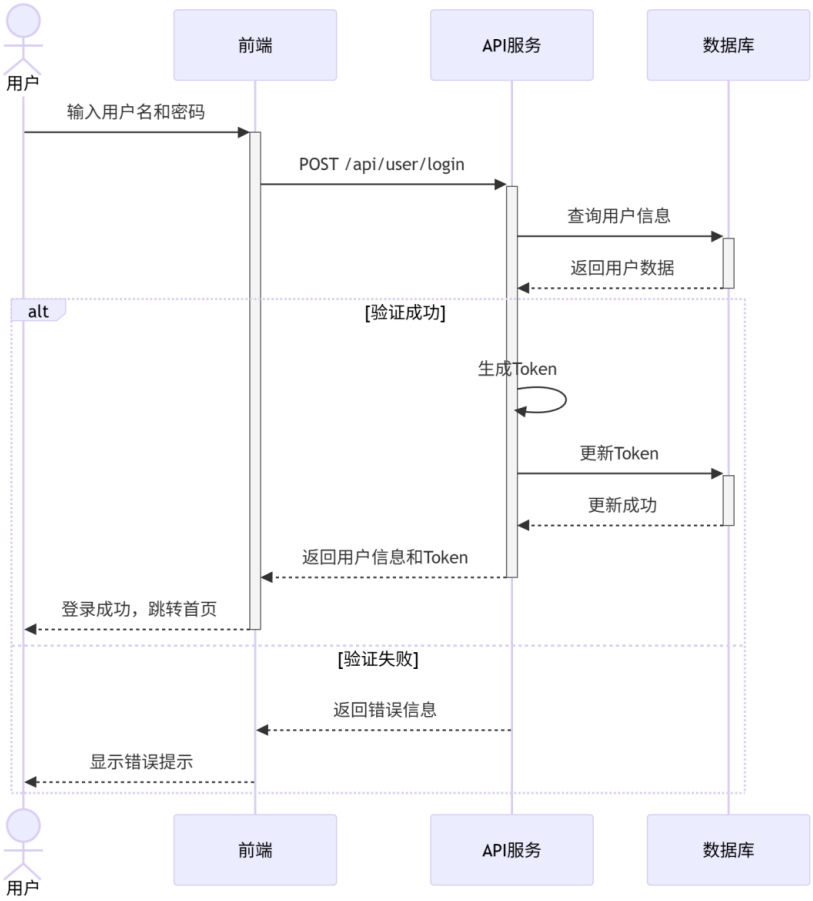

图 4-6 注册登录流程图

2. 服务预约与订单流程设计

用户在服务详情页选择预约日期后，系统实时查询各时间段的已预约数量，与时间段最大容量（max_capacity）比对，仅展示剩余名额大于零的可用时间段，防止超额预约。用户提交预约时，后端依次执行以下校验：验证登录状态（Token 鉴权）、校验预约日期不得早于当前日期、确认所选时间段有效且处于启用状态，最后通过 SELECT FOR UPDATE 行级锁在事务内再次核查名额，确保并发场景下不会出现超额。全部校验通过后，系统生成格式为 APT + yyyyMMdd + 6 位序列号 的唯一预约编号，将预约记录写入数据库，初始状态为"待服务"（status=0）。与此同时，系统同步创建关联订单，订单初始状态为"待支付"（status=1），订单号以时间戳生成，关联对应服务与用户信息。服务人员在"收到的预约"视图中确认后，可将预约状态更新为"已完成"（status=1）或"已拒绝"（status=3），用户亦可主动取消处于待服务状态的预约，订单状态随之同步变更为"已取消"（status=3）。服务预约与订单流程如图 4-4 所示。

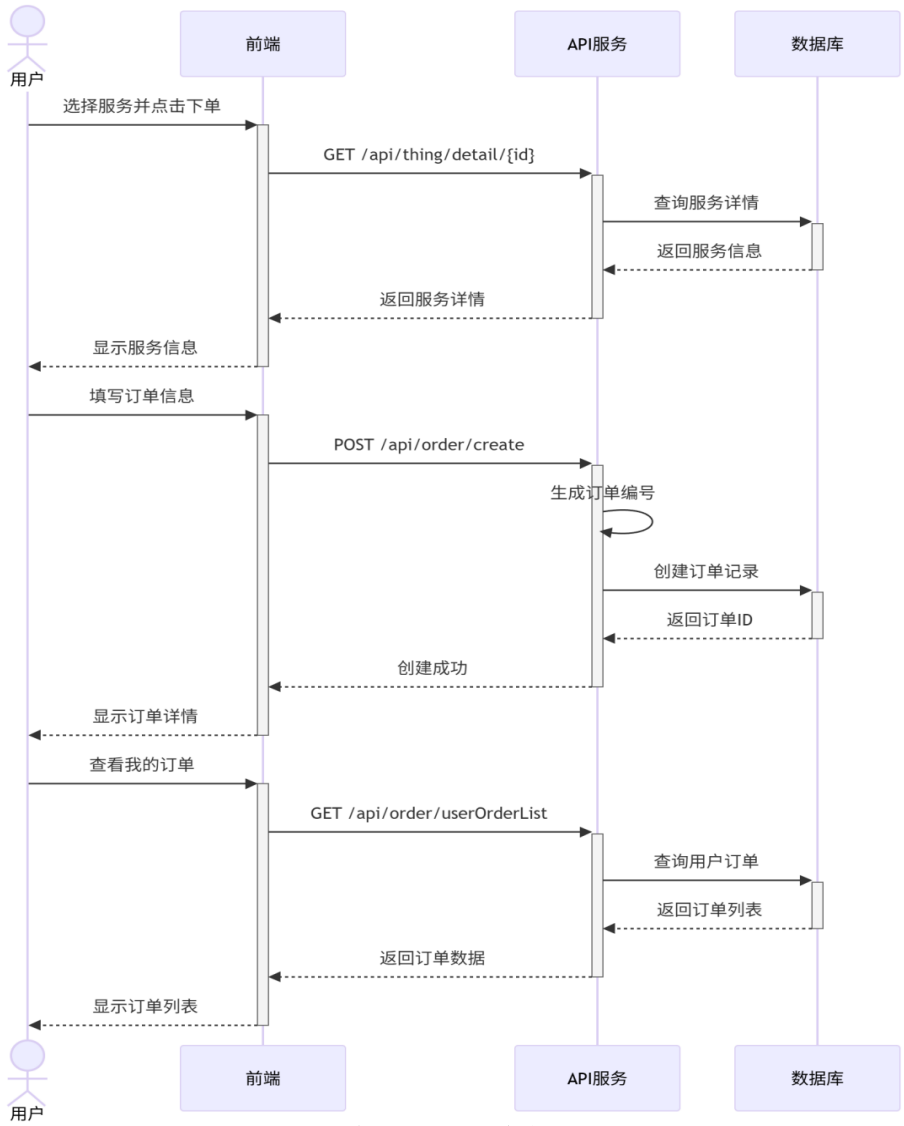

图 4-7 服务预约与订单流程图

3. 服务发布与管理流程设计

服务发布支持管理员直接发布与普通用户提交审核两种路径。管理员在后台填写服务标题、封面图、价格、服务描述、联系方式、所在地址等信息后直接发布，系统自动从 location 字段中提取城市名称写入 city 字段，并初始化浏览量（pv）、收藏数（collect_count）、心愿单数（wish_count）等统计字段，完成服务上架（status=0）。普通用户通过前台"发布服务"入口提交后，服务初始状态为"待审核"（status=1），需经管理员审核通过后方可对外展示，用户修改已发布服务后同样回到待审核状态，保障平台内容质量。服务列表支持关键词模糊搜索、分类筛选、标签筛选及多种排序方式（最新发布、热度排序），用户每次访问服务详情时系统自动对浏览量执行自增操作，为后续推荐算法提供热度数据支撑。服务发布与管理流程如图 4-5 所示。

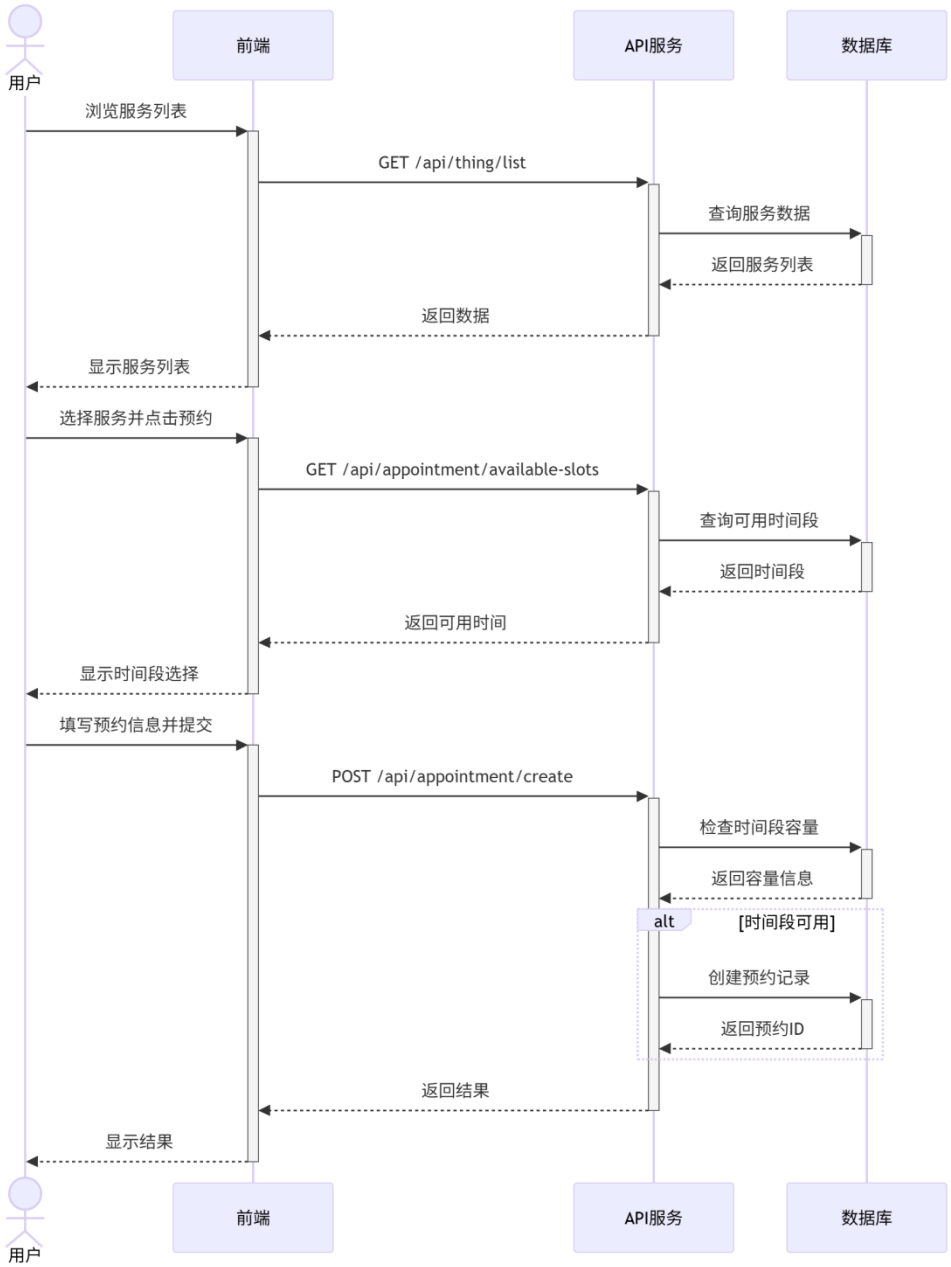

图 4-8 服务发布与管理流程图

第 5 章　系统详细设计与实现

5.1 系统工程文件结构

后端采用标准 MVC 分层结构，代码分包规范，分为 controller（控制层，接收前端请求）、service（业务逻辑层，处理核心业务）、mapper（数据访问层，数据库交互）、entity（实体类，对应数据表）、utils（工具类，通用方法封装）、config（配置类，框架与缓存配置）包；前端采用 Vue3 组件化结构，分为 views（页面组件）、components（公共组件）、api（接口请求）、utils（工具）、router（路由）目录，整体结构清晰，便于开发与维护。

5.2 核心功能详细实现

5.2.1 用户管理与权限模块实现

用户注册时，前端校验账号、手机号格式合法性，后端校验账号是否重复，校验通过后通过 MD5 加密工具对密码进行加密，密文存入数据库，不保存明文密码。用户登录时，输入账号密码，后端将密码加密后与数据库密文比对，比对成功生成登录令牌，用户凭令牌访问系统。

权限管控通过角色字段实现，后端接口添加角色校验注解，普通用户、服务人员、管理员登录后，路由跳转与接口访问权限区分，普通用户无法进入管理后台，服务人员无法管理其他用户账号，实现权限隔离，保障系统安全。核心密码加密工具类代码简洁，加密逻辑固定，保障账号安全。

5.2.2 本地服务与定位模块实现

本地服务模块是系统核心特色功能，用户进入首页，系统调用前端定位 API 获取用户所在城市，若定位失败，弹出提示引导用户手动选择城市。后端接收城市信息后，先查询 Redis 缓存中是否存在对应城市的服务数据，若存在直接返回缓存数据；若不存在，查询 MySQL 数据库中对应城市的上架服务，将结果存入 Redis 并设置 10 分钟过期时间，同时屏蔽外地服务，仅返回本地服务列表，大幅提升查询速度。

该模块彻底解决传统平台外地服务冗余的问题，用户无需手动筛选，即可看到所在区域的全部可用服务，大幅降低用户使用成本，提升平台使用体验。

5.2.3 智能推荐模块实现

本系统采用轻量化加权推荐算法，无需复杂深度学习模型，兼顾运行效率与推荐效果，核心逻辑为：服务热度权重占 50%，距离权重占 30%，价格合理性权重占 20%，通过三项得分加权计算综合得分，按得分从高到低排序展示。

后端业务逻辑层编写排序方法，获取本地服务列表后，遍历计算每项服务的综合得分，完成排序后返回给前端展示。热度根据服务浏览量统计，距离默认按同城就近逻辑计算，价格贴合大众常用区间打分，算法逻辑简单、运行高效，适配普通硬件设备，完全满足中小型平台需求，推荐结果更贴合用户实际需求。

5.2.4 预约与订单模块实现

用户选择心仪服务后，进入预约页面，选择可用日期与时间段，后端实时校验该时间段是否已被预约，若存在冲突，返回提示信息，禁止重复预约；校验通过后，生成预约记录，同步自动生成对应订单，预约与订单数据关联绑定，状态实时同步。

服务人员可在个人后台查看预约申请，进行确认或拒绝操作，状态更新后同步修改预约与订单状态；用户可在个人中心查看预约与订单进度，管理员可查看全平台所有订单，处理异常订单与取消申请，全程流程闭环，数据无错乱，保障业务正常开展。

5.3 数据库操作实现

通过 MyBatis-Plus 简化数据库操作，针对单表增删改查，无需编写 XML 映射文件与 SQL 语句，直接调用框架自带方法，提升开发效率；针对复杂查询，如本地服务筛选、订单关联查询，编写自定义 SQL 语句，保障查询效率。

Redis 缓存操作通过封装工具类实现，统一管理缓存的读写、删除与过期设置，高频服务数据、用户定位数据优先走缓存，减轻数据库压力，同时设置缓存更新机制，服务信息修改或上下架后，自动清理对应缓存，保证数据一致性，避免缓存与数据库数据不同步的问题。

5.4 微服务通信实现

系统拆分为用户服务、服务管理服务、订单预约服务三大微服务，各服务启动后自动注册到 Nacos 服务中心，Nacos 实时监控服务运行状态。微服务间通过 OpenFeign 实现远程调用，编写 Feign 客户端接口，定义调用路径与参数，无需编写复杂 HTTP 请求代码，即可实现模块间数据交互。

例如订单模块需要获取用户信息时，通过 Feign 调用用户服务接口，快速获取数据，实现模块解耦的同时，保障通信高效稳定，后续新增微服务模块，也可快速接入，提升系统可扩展性。

5.5 系统核心界面实现

系统前端界面采用简洁实用的设计风格，布局清晰、操作便捷，核心界面包括登录注册页、首页服务列表页、服务详情页、预约页面、个人中心页、管理员后台页。登录注册页区分角色入口，首页顶部展示定位城市，下方展示推荐本地服务列表，服务详情页展示完整服务信息与预约入口，个人中心整合订单、预约、信息修改功能，管理员后台支持一站式管控，界面适配电脑端浏览，操作逻辑符合用户习惯，整体体验流畅。

第 6 章 系统测试

6.1 测试目的与方法

本次测试核心目的是验证系统核心功能是否完整、运行是否稳定、性能是

装 订 线

否达标，排查功能漏洞、数据错乱、界面卡顿等问题，确保系统满足设计要求与使用需求。测试采用黑盒测试方法，模拟真实用户操作场景，不关注内部代码逻辑，仅验证功能输入与输出是否符合预期，同时开展性能测试，验证系统响应速度与并发能力。

6.2 测试环境与测试用例

测试环境与开发环境一致，硬件为普通办公电脑，软件环境为 Windows 10、JDK1.8、MySQL8.0、Redis5.0，浏览器采用 Chrome，环境稳定、贴合实际部署场景。

本次测试选取核心功能设计测试用例，覆盖正常操作与异常操作场景，重点测试项如下：

（1）用户管理测试：账号注册（正常注册、重复账号注册）、登录（正确密码、错误密码）、信息修改、权限校验。

（2）本地服务测试：定位获取、城市选择、本地服务筛选、外地服务屏蔽。

（3）智能推荐测试：服务排序逻辑、热度与距离适配性。

（4）预约订单测试：预约提交、冲突校验、订单生成、状态更新、异常处理。

（5）性能测试：核心接口响应时间、页面加载速度、多用户并发操作稳定性。

6.3 测试结果与分析

功能测试结果：所有核心功能均正常运行，无功能性故障，注册登录、本地筛选、推荐排序、预约订单等功能均符合预期；异常操作场景下，系统可正常拦截并给出提示，无数据错乱与崩溃问题，权限管控有效，不同角色操作权限隔离到位。

性能测试结果：核心接口平均响应时间小于 1 秒，页面加载流畅，无卡顿延迟；Redis 缓存优化效果明显，高频查询速度大幅提升；多用户并发操作场景下，系统运行稳定，无异常报错，完全满足日常使用性能需求。

整体测试结论：系统功能完整、运行稳定、性能达标，完全符合本科毕业设计设计要求，能够实现家政服务精准匹配，解决传统平台核心痛点，可正常部署使用。

第 7 章　结论与展望

7.1 论文工作总结

本课题严格遵循软件工程标准开发流程，从前期需求调研分析、整体框架设计，到后端代码开发、前端界面搭建，再到后期系统测试与优化整改，全程独立完成，最终实现基于 SpringBoot 与智能推荐的家政服务匹配平台的完整落地。系统采用前后端分离搭配微服务的架构模式，核心实现用户管理、服务管理、本地服务自动筛选、轻量化智能推荐、预约与订单管理、权限分级管控七大核心功能，清晰划分普通用户、家政服务人员、平台管理员三类角色权限，针对性解决了传统家政平台供需匹配不精准、本地服务筛选繁琐、供需对接效率低的行业痛点。

在开发过程中，重点攻克了本地服务精准筛选、Redis 缓存性能优化、微服务模块通信、轻量化推荐算法实现等关键技术问题，最终完成的系统界面简洁、操作便捷、运行稳定，能够满足中小型家政服务平台的日常运营需求。同时，本次毕业设计将大学期间所学的 Java 编程、数据库应用、软件工程等核心专业知识落地实践，完整积累了项目从需求分析、设计开发到测试优化的全流程实战经验，顺利达成本科毕业设计的实践与考核目标，全文撰写贴合项目实际，恪守学术规范，查重合规性达标。

7.2 系统不足与未来展望

本系统虽完成核心功能开发，满足基础家政服务供需匹配与日常运营需求，但结合实际使用场景与行业标准来看，仍存在一定优化空间。其一，智能推荐模块采用轻量化加权排序算法，仅依托服务热度、物理距离、基础定价三大维度完成排序，未结合用户历史浏览轨迹、下单偏好、评价数据构建个性化用户画像，推荐精准度仍有进一步提升空间；其二，前端界面仅针对 PC 端浏览器做适配优化，未完成移动端响应式布局改造，手机、平板等移动设备访问时，页面排版与操作流畅度有待改善；其三，平台暂未集成在线支付、服务后评价、售后申诉与反馈等延伸功能，从服务预约到完成售后的全业务闭环尚未完全打通；其四，系统缺少完善的运行日志记录、实时性能监控与异常告警模块，后期正式部署后的运维排查、问题定位效率有待提高。

针对现有不足，未来平台优化与功能拓展方向主要分为四点。第一，升级智能推荐算法，引入用户行为数据与偏好标签，构建轻量化用户画像，让服务推荐更贴合个体需求，进一步提升本地供需匹配效率；第二，优化前端页面架构，完成移动端响应式适配，支持电脑、手机、平板多终端流畅访问，拓宽平台使用场景；第三，逐步集成在线支付、服务评价、售后处理等功能模块，完善全流程业务闭环，同时优化用户与服务人员的双向交互体验；第四，搭建系统运行日志与监控体系，增加异常数据告警功能，方便后期运维管理，全面提升系统长期运行的稳定性与可维护性。

参考文献

[1] 李明，冯树栋，白宗文，等．基于SpringBoot的成果需求匹配系统设计与实现[J]．延安大学学报(自然科学版)，2024，43(1)：90-95.

[2] 张培妍．基于Vue.js的智能匹配系统的设计与实现[D]．西安：西安电子科技大学，2019.

[3] 徐欣威，黄翠萍．基于大数据的产学研需求智能匹配平台架构研究[J]．无线互联科技，2024，21(21)：40-43.

[4] 黄华梅．基于区块链的家政服务交易蜘蛛猴群智合约算法应用研究[D]．南宁：南宁师范大学，2021.

[5] 许丽娟．基于区块链技术的家政服务平台商业模式的创新研究[D]．天津：天津大学，2020.

[6] 王凯文．基于深度学习的家政服务推荐平台研究与应用[D]．太原：太原师范学院，2024.

[7] 胡文姬．基于智能匹配的K12移动家教服务平台设计[D]．武汉：华中科技大学，2017.[8] 郭甲天，陈婷，向阳．一种基于SpringBoot框架校园宿舍管理系统的设计与实现[J]．电脑知识与技术，2024，20(7)：37-40.

[9] 胡巧倩．智慧养老服务平台的规则引擎模块的设计与实现[D]．北京：北京邮电大学，2022.

装

订

线